

Defensa del Proyecto – DWEC T06P01

Harry Potter App · Arroyo Mikel 25-26

Guía completa de defensa: qué hace cada cosa y por qué

Índice

1. [Estructura del proyecto](#)
 2. [Conceptos clave antes de empezar](#)
 3. [Ejercicio 1 – La aplicación base](#)
 4. [Ejercicio 2 – APIs del navegador](#)
 5. [Ampliación – Frases por personaje](#)
 6. [Ampliación – Búsqueda avanzada por año](#)
 7. [Gestión de errores: los 4 tipos](#)
 8. [Preguntas trampa del profesor](#)
-

1. Estructura del proyecto

```
dwec_t06p01_arroyo_mikel/
|
├── dist/                               ← Carpeta con los archivos HTML que abre el
navegador
|   ├── index.html                     ← Página principal (una sola página con todas las
secciones)
|   ├── frases.html                   ← Página "Frases por personaje" (Ampliación)
|   └── busqueda-avanzada.html        ← Página "Búsqueda avanzada" (Ampliación)
|
├── js/                                ← Todo el JavaScript del proyecto
|   ├── ejercicio01_dom.js            ← Lógica de index.html (Ejercicios 1 y 2)
|   ├── frases.js                     ← Lógica de frases.html
|   └── busqueda-avanzada.js          ← Lógica de busqueda-avanzada.html
|
├── css/
|   └── micss.css                     ← Estilos propios que complementan Bootstrap
|
└── *.pug                             ← Plantillas fuente (se compilaron para generar
el HTML)
```

Por qué está separado en `dist/` y `js/`

Los archivos HTML están en `dist/` porque es la carpeta de salida de la compilación de Pug. Los archivos JS están en `js/` y se referencian desde el HTML con rutas relativas:

```
<!-- En index.html, la ruta ../js/ sube un nivel desde dist/ hasta la raíz -->
<script src="../js/ejercicio01_dom.js"></script>
```

`../` significa "sube un nivel en la carpeta". Desde `dist/index.html`, `../js/` apunta a la carpeta `js/` de la raíz.

2. Conceptos clave antes de empezar

2.1 ¿Qué es una API y qué es la HP-API?

Una **API** (Application Programming Interface) es un servidor web al que le mandas una petición (una URL) y te devuelve datos. La HP-API de Harry Potter es pública (no necesita contraseña) y devuelve datos en formato **JSON**.

URL base: <https://hp-api.onrender.com>

Endpoints usados en el proyecto:

URL	Qué devuelve
/api/characters	Array con TODOS los personajes
/api/characters/house/gryffindor	Array con personajes de Gryffindor
/api/character/{id}	Array con UN personaje por su ID

Ejemplo de lo que devuelve la API (un personaje):

```
{
  "id": "9e3f7ce4-b9a7-4244-b709-dae5c1f1d4a8",
  "name": "Harry Potter",
  "house": "Gryffindor",
  "species": "human",
  "patronus": "stag",
  "yearOfBirth": 1980,
  "alive": true,
  "image": "https://ik.imagekit.io/hpapi/harry.jpg",
  "gender": "male"
}
```

2.2 ¿Qué es `fetch` y por qué lo necesitamos?

`fetch` es la función de JavaScript moderna para hacer peticiones a una API. Es como decirle al navegador: "ve a esta URL, trae los datos y cuando lleguen, sigue ejecutando el código".

El problema sin `fetch`: No podemos hacer una petición HTTP directamente con JavaScript normal porque JavaScript es **síncrono** por defecto (ejecuta una línea, espera a que termine, pasa a la siguiente). Si

esperáramos a que llegaran los datos de la API bloqueando el navegador, la página se quedaría congelada.

La solución: asincronía. Con `fetch` + `async/await`, el navegador hace la petición "en segundo plano" y sigue pintando la página. Cuando llegan los datos, los procesa.

2.3 ¿Qué es `async/await`? Explicación fácil

Analogía: Imagina que pides una pizza por teléfono. Tienes dos opciones:

1. **Síncrono:** Te quedas pegado al teléfono sin hacer nada hasta que llega la pizza. La página se congela.
2. **Asíncrono:** Cuelgas y sigues haciendo cosas. Cuando llaman al timbre (llegan los datos), atiendes.

`async/await` es la forma de escribir código asíncrono que **parece síncrono** (se lee de arriba a abajo) pero no bloquea el navegador.

```
// SIN async/await (forma antigua con .then):
fetch("https://hp-api.onrender.com/api/characters")
  .then(respuesta => respuesta.json())
  .then(datos => console.log(datos))
  .catch(error => console.error(error));

// CON async/await (forma moderna, mucho más legible):
async function cargarDatos() {
  const respuesta = await fetch("https://hp-api.onrender.com/api/characters");
  const datos = await respuesta.json();
  console.log(datos);
}
```

`await` significa "espera aquí a que esto termine antes de continuar". Pero al ser `async`, el navegador no se bloquea: simplemente "pausa" esa función y sigue haciendo otras cosas.

Regla importante: Solo puedes usar `await` dentro de una función marcada con `async`.

2.4 ¿Qué es el DOM y qué es manipularlo?

El **DOM** (Document Object Model) es la representación en memoria de todo el HTML de la página. Cuando escribes `document.getElementById("mi-id")`, estás accediendo a ese árbol de objetos y puedes modificarlo.

Manipular el DOM = cambiar el contenido de la página sin recargarla (añadir filas a una tabla, mostrar/ocultar elementos, cambiar texto...).

3. Ejercicio 1 – La aplicación base

3.1 El HTML base (`dist/index.html`)

La práctica pide **una sola página** con varias secciones. Esto se consigue poniendo todas las secciones dentro del mismo `<main>`:

```
<main class="container my-4">

  <section id="buscador">...</section>      <!-- Sección 1 -->
  <hr class="my-5">
  <section id="seccion-bienvenida">...</section> <!-- Sección 2 -->
  <hr class="my-5">
  <section id="seccion-favoritos">...</section> <!-- Sección 3 -->
  <hr class="my-5">
  <section id="seccion-quienes-somos">...</section> <!-- Sección 4 -->

</main>
```

El **navbar** usa anclas (el símbolo #) para navegar a cada sección SIN recargar la página:

```
<a class="nav-link" href="#buscador">Buscador</a>
<!-- Al pulsar, el navegador hace scroll hasta el elemento con id="buscador" -->
```

¿Por qué **sticky-top** en el navbar?

```
<nav class="navbar navbar-expand-lg navbar-dark bg-secondary sticky-top">
```

sticky-top hace que el navbar se quede fijo en la parte superior mientras haces scroll. Sin esta clase, el navbar desaparecería al bajar.

¿Por qué **scroll-margin-top: 70px** en el CSS?

```
section {
  scroll-margin-top: 70px;
}
```

Sin esto, al hacer click en "Buscador" el navbar sticky tapanía el título de la sección. Este margen empuja el punto de llegada del scroll 70px hacia abajo, evitando la superposición.

3.2 El **DOMContentLoaded** y la carga inicial

```
// js/ejercicio01_dom.js

document.addEventListener("DOMContentLoaded", async () => {
  // Todo el código de inicialización va aquí dentro
});
```

¿Por qué **DOMContentLoaded**? Cuando el navegador carga una página, primero descarga el HTML y lo convierte en DOM, luego ejecuta el JS. Si ejecutáramos el JS antes de que el DOM exista, `document.getElementById("form-busqueda")` devolvería `null` (no encuentra nada). Con **DOMContentLoaded**, garantizamos que el DOM ya está listo antes de ejecutar nada.

¿Por qué **async** en el **callback**? Porque dentro necesitamos usar `await` con `fetch`. Sin **async**, el `await` daría error de sintaxis.

3.3 La petición a la API y la variable `datos`

```
const API_URL = "https://hp-api.onrender.com/api/characters";

document.addEventListener("DOMContentLoaded", async () => {
  let datos = []; // Array vacío donde guardaremos TODOS los personajes

  try {
    const respuesta = await fetch(API_URL); // 1. Pide los datos a la API

    if (!respuesta.ok) { // 2. Comprueba si la respuesta es un error HTTP
      throw new Error(`Error HTTP ${respuesta.status}: ${respuesta.statusText}`);
    }

    datos = await respuesta.json(); // 3. Convierte el JSON a array de objetos JS

    cargarSeccionBienvenida(datos, loader, contenedorTarjetas); // 4. Usa los datos

  } catch (error) {
    // 5. Si algo falla, muestra el error
  }
});
```

El flujo paso a paso:

1. `fetch(API_URL)` → el navegador hace una petición GET a la URL. Devuelve una "promesa" de respuesta.
2. `await fetch(...)` → espera a que llegue la respuesta (los headers HTTP).
3. `respuesta.ok` → es `true` si el código HTTP está entre 200-299. Si la API devuelve 404 o 500, es `false`.
4. `respuesta.json()` → lee el cuerpo de la respuesta y lo parsea como JSON. También es `async` (por eso lleva `await`).
5. `datos = await respuesta.json()` → ahora `datos` es un array de objetos JavaScript.

¿Por qué guardamos los datos en una variable fuera del `try`?

```
let datos = []; // Declarada FUERA del try/catch
```

```
try {
  datos = await respuesta.json(); // Asignada DENTRO
}

// Aquí abajo también podemos usar "datos"
formulario.addEventListener("submit", () => {
  realizarBusqueda(datos, ...); // <-- Puede acceder a datos
});
```

Si la declaráramos dentro del `try`, solo existiría dentro de ese bloque. Al declararla fuera, todos los event listeners del mismo `DOMContentLoaded` pueden acceder a ella. Esto es el **ámbito de variables** (scope).

3.4 El Buscador – dos eventos, un propósito

```
// Búsqueda en tiempo real (mientras escribes)
input.addEventListener("input", () => {
  realizarBusqueda(datos, input, tablaBody, tablaContenedor, zonaErrores,
false);
  //
  ^^^^
  //
  submit
});

// Búsqueda al pulsar el botón
formulario.addEventListener("submit", (event) => {
  event.preventDefault(); // <-- IMPORTANTE
  realizarBusqueda(datos, input, tablaBody, tablaContenedor, zonaErrores, true);
  //
  //
  submit
});
```

false = no es
true = es

`event.preventDefault()` — esto es crucial. Un `<form>` en HTML, al hacer submit, por defecto recarga la página enviando los datos al servidor. Como esta aplicación no tiene servidor y maneja todo en el cliente, necesitamos cancelar ese comportamiento. Sin `preventDefault`, la página se recargaría y perderíamos todos los datos cargados.

El **parámetro** `esSubmit` controla cuándo mostrar el error de "campo vacío":

```
const realizarBusqueda = (... , esSubmit) => {
  const texto = input.value.trim().toLowerCase();

  if (texto === "") {
    if (esSubmit) { // Solo muestra error si es un submit explícito
      mostrarError(zonaErrores, "El campo de búsqueda no puede estar
vacío.");
    }
  }
}
```

```
        return; // Para el resto del código (no busca nada vacío)
    }
    // ...
};
```

¿Por qué? Porque cuando el evento `"input"` se dispara al empezar a escribir la primera letra, el campo pasa por estar vacío. Si mostráramos el error siempre, aparecería al abrir la página o al borrar. Solo tiene sentido mostrar "campo vacío" si el usuario hace click en Buscar conscientemente.

`.trim()` elimina espacios al inicio y al final: `" Harry ".trim() → "Harry"`. Así, escribir solo espacios cuenta como vacío.

`.toLowerCase()` convierte a minúsculas para que la búsqueda no sea sensible a mayúsculas: buscar "harry" encuentra "Harry Potter".

3.5 Filtrado del array con `.filter()`

```
const filtrados = datos.filter((p) => p.name.toLowerCase().includes(texto));
```

`.filter()` recorre el array y devuelve un **nuevo array** solo con los elementos que cumplen la condición. Para cada personaje `p`, evalúa si `p.name.toLowerCase().includes(texto)`.

Ejemplo concreto:

```
// datos = [{name: "Harry Potter", ...}, {name: "Hermione Granger", ...}, {name:
"Ron Weasley", ...}]
// texto = "harry"

const filtrados = datos.filter((p) => p.name.toLowerCase().includes("harry"));
// filtrados = [{name: "Harry Potter", ...}]
// Solo devuelve los que tienen "harry" en el nombre (en minúsculas)
```

`.includes(texto)` devuelve `true` si la cadena contiene el texto buscado en cualquier posición. Así, buscar "pot" encontraría "Harry Potter" y "Neville Longbottom" (tiene "ott" en "Longbottom"... espera, ese no. Pero "potter" encontraría "Harry Potter" y "James Potter").

3.6 Creación de la tabla con nodos del DOM

El enunciado especifica explícitamente que **la tabla se hará creando nodos del DOM**. Esto significa NO usar `innerHTML` para las filas:

```
filtrados.forEach((p) => {
    // Crear el elemento <tr>
    const fila = document.createElement("tr");
```

```

// --- Celda de foto ---
const celdaFoto = document.createElement("td");
if (p.image) {
  const img = document.createElement("img");
  img.src = p.image;          // Atributo src del <img>
  img.alt = p.name;           // Accesibilidad: texto alternativo
  img.width = 50;
  img.height = 50;
  img.style.objectFit = "cover"; // La imagen no se deforma
  img.className = "img-thumbnail rounded"; // Clases Bootstrap
  celdaFoto.appendChild(img); // Añadir <img> dentro de <td>
} else {
  celdaFoto.innerHTML = '<span class="text-muted small">Sin foto</span>';
}

// --- Celda de nombre ---
const celdaNombre = document.createElement("td");
celdaNombre.textContent = p.name; // textContent es más seguro que innerHTML
//                               (evita inyección HTML/XSS)

// --- Celda de casa ---
const celdaCasa = document.createElement("td");
celdaCasa.textContent = p.house || "Sin casa";
//                               ^^ si p.house es "", null o undefined → "Sin
casa"

// --- Celda de acción (botón favorito) ---
const celdaAccion = document.createElement("td");
const btnFav = document.createElement("button");
actualizarBotonFavorito(btnFav, p); // Configurar el botón
btnFav.addEventListener("click", () => {
  toggleFavorito(p);                // Toggle en localStorage
  actualizarBotonFavorito(btnFav, p); // Actualizar apariencia del botón
  cargarFavoritos();                 // Refrescar la sección de favoritos
});
celdaAccion.appendChild(btnFav);

// Añadir todas las celdas a la fila
fila.append(celdaFoto, celdaNombre, celdaCasa, celdaAccion);
// fila.append() acepta múltiples argumentos a la vez

// Añadir la fila al <tbody> de la tabla
tablaBody.appendChild(fila);
});

```

textContent vs innerHTML — importante saberlo:

- **textContent** = inserta el texto tal cual, sin interpretar HTML. Seguro contra inyección.
- **innerHTML** = interpreta las etiquetas HTML. Útil pero peligroso si el texto viene del usuario (podría inyectar `<script>`).

- Para texto de la API (datos externos) se usa `textContent`. Para plantillas HTML propias es aceptable `innerHTML`.

`fila.append()` vs `fila.appendChild()`:

- `appendChild(elemento)` → añade UN solo elemento.
- `append(elem1, elem2, ...)` → añade MÚLTIPLES a la vez (más moderno).

3.7 La sección Bienvenida: spinner + tarjetas Bootstrap

```
const cargarSeccionBienvenida = (todos, loader, contenedor) => {

  if (loader) loader.style.display = "block"; // Mostrar el spinner
  contenedor.innerHTML = ""; // Limpiar contenido anterior

  setTimeout(() => { // Simular espera de red
    if (loader) loader.style.display = "none"; // Ocultar spinner

    const casas = ["Gryffindor", "Slytherin", "Hufflepuff", "Ravenclaw"];
    let html = ""; // Acumular el HTML de todas las tarjetas

    casas.forEach((casa) => {
      // Filtrar personajes de esta casa
      const grupo = todos.filter((p) => p.house === casa);

      // Seleccionar 2 aleatorios
      const aleatorios = grupo.sort(() => 0.5 - Math.random()).slice(0, 2);

      aleatorios.forEach((p) => {
        const foto = p.image || "https://placeholder.co/300x200?";
        text=Sin+foto";
        // Si no tiene imagen, usa una imagen de marcador de posición

        // Template String: construir HTML con los datos del personaje
        html += `
          <div class="col-12 col-sm-6 col-md-4 col-lg-3">
            <div class="card h-100 shadow-sm">
              
              <div class="card-body">
                <h5 class="card-title text-success">${p.name}</h5>
                <ul class="list-unstyled small mb-0">
                  <li><strong>Casa:</strong> ${p.house || "Sin
casa"}</li>
                  <li><strong>Especie:</strong> ${p.species ||
"Desconocida"}</li>
                  <li><strong>Patronus:</strong> ${p.patronus ||
"Desconocido"}</li>
                  <li><strong>Año:</strong> ${p.yearOfBirth ||
"Desconocido"}</li>
```

```

        </ul>
      </div>
    </div>
  </div>
  `;
});
});

contenedor.innerHTML = html; // Insertar todas las tarjetas de golpe

}, 2000); // 2 segundos de "espera simulada"
};

```

¿Por qué Template Strings y no nodos del DOM? El enunciado dice "las tarjetas se dibujarán usando Template Strings". Son más cómodos para estructuras HTML complejas. Los Template Strings (comillas invertidas ```) permiten:

- Texto multilínea sin `\n`
- Interpolar variables con `${variable}`

`sort(() => 0.5 - Math.random())` — cómo baraja el array: `Array.sort()` necesita una función que compare dos elementos y devuelva positivo, negativo o cero. Al devolver `0.5 - Math.random()`, el resultado es un número aleatorio entre -0.5 y 0.5. Con positivo, el orden se mantiene; con negativo, se invierte. El resultado final es un orden aleatorio.

`.slice(0, 2)` — devuelve los primeros 2 elementos del array ya mezclado. No modifica el original.

`setTimeout(() => { ... }, 2000)` — ejecuta el código del interior después de 2000ms (2 segundos). El enunciado pide simular una espera para mostrar el spinner. El spinner aparece inmediatamente y desaparece cuando se ejecuta el callback del `setTimeout`.

Clases Bootstrap para el responsive:

<code>col-12</code>	→ pantallas muy pequeñas: 1 tarjeta por fila (100% de ancho)
<code>col-sm-6</code>	→ pantallas pequeñas (≥576px): 2 tarjetas por fila (50% cada una)
<code>col-md-4</code>	→ pantallas medianas (≥768px): 3 tarjetas por fila (33% cada una)
<code>col-lg-3</code>	→ pantallas grandes (≥992px): 4 tarjetas por fila (25% cada una)

El sistema de grid de Bootstrap tiene 12 columnas. `col-lg-3` ocupa 3 de 12, es decir, 25%.

4. Ejercicio 2 – APIs del navegador

4.1 Aviso de cookies con `sessionStorage`

¿Qué es `sessionStorage`? Es una "caja" del navegador donde podemos guardar datos de texto. Se borra automáticamente cuando el usuario cierra la pestaña o el navegador. No va al servidor, es solo del cliente.

```
<!-- En index.html: el banner empieza oculto con d-none -->
<div id="aviso-cookies" class="d-none fixed-bottom bg-dark text-white p-3 shadow-
lg">
  <div class="container d-flex ...">
    <span>Esta aplicación utiliza almacenamiento de sesión...</span>
    <button id="btn-aceptar-cookies" class="btn btn-warning btn-sm">
      Aceptar y continuar
    </button>
  </div>
</div>
```

```
const gestionarCookies = () => {
  const banner = document.getElementById("aviso-cookies");
  const btnAceptar = document.getElementById("btn-aceptar-cookies");

  // sessionStorage.getItem devuelve null si la clave no existe
  if (!sessionStorage.getItem("cookiesAceptadas")) {
    // Primera vez en esta sesión: mostrar el banner
    banner.classList.remove("d-none"); // quitar la clase que lo oculta
  }
  // Si ya aceptó, getItem devuelve "true" (string), !("true") = false → no
  entra

  btnAceptar.addEventListener("click", () => {
    sessionStorage.setItem("cookiesAceptadas", "true"); // guardar aceptación
    banner.classList.add("d-none"); // ocultar banner
  });
};
```

Flujo completo:

1. Usuario abre la página por primera vez → `getItem("cookiesAceptadas")` devuelve `null` → `!null = true` → entra en el `if` → muestra el banner.
2. Usuario pulsa "Aceptar" → `setItem("cookiesAceptadas", "true")` guarda en memoria → banner se oculta.
3. Usuario navega o recarga → `getItem("cookiesAceptadas")` devuelve `"true"` → `!("true") = false` → no entra en el `if` → el banner nunca aparece.
4. Usuario cierra la pestaña → `sessionStorage` se borra → al volver a abrir, vuelve al paso 1.

`classList.remove("d-none")` vs `classList.add("d-none")`:

- `d-none` es una clase de Bootstrap que aplica `display: none` (elemento invisible y no ocupa espacio).
- `remove("d-none")` → quita esa clase → el elemento se hace visible.
- `add("d-none")` → añade esa clase → el elemento se oculta.
- Es más limpio que cambiar `element.style.display` directamente.

¿Por qué `fixed-bottom`? La clase `fixed-bottom` de Bootstrap posiciona el elemento al fondo de la pantalla, fijo (no se mueve al hacer scroll), ocupando todo el ancho. Así el banner siempre es visible sin molestar el

contenido principal.

4.2 Geolocalización + Mapa Leaflet

¿Qué es Leaflet? Leaflet es una librería JavaScript de código abierto para mostrar mapas interactivos. Usa datos de OpenStreetMap (gratuito, sin clave API). Se carga desde CDN:

```
<!-- CSS de Leaflet (en el <head>) -->
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">

<!-- JS de Leaflet (antes del script propio) -->
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script src="../js/ejercicio01_dom.js"></script>
```

Es importante que el JS de Leaflet vaya ANTES del nuestro, porque nuestro código usa el objeto `L` que define Leaflet.

```
const inicializarMapa = () => {
  const contenedorMapa = document.getElementById("mapa");

  // Comprobación defensiva: si no hay div del mapa o Leaflet no cargó, salir
  if (!contenedorMapa || typeof L === "undefined") return;

  // Coordenadas del IES (latitud, longitud)
  const latIES = 43.2630;
  const lngIES = -2.9350;

  // Crear el mapa dentro del div #mapa, centrado en el IES con zoom 15
  const mapa = L.map("mapa").setView([latIES, lngIES], 15);

  // Añadir la capa de tiles de OpenStreetMap (las "losetas" del mapa)
  L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
    attribution: '© OpenStreetMap contributors',
    maxZoom: 19,
  }).addTo(mapa);

  // Marcador en el IES con un popup
  L.marker([latIES, lngIES])
    .addTo(mapa)
    .bindPopup("<strong>IES</strong><br>Centro educativo")
    .openPopup(); // Abrir el popup automáticamente al cargar

  const msgGeo = document.getElementById("msg-geolocalizacion");

  // Comprobar si el navegador soporta Geolocalización
  if (!navigator.geolocation) {
    if (msgGeo) msgGeo.textContent = "Tu navegador no soporta geolocalización.";
    return;
  }
}
```

```

    }

    // Pedir la ubicación al usuario
    navigator.geolocation.getCurrentPosition(
        // Callback de ÉXITO (si el usuario acepta)
        (pos) => {
            const { latitude, longitude } = pos.coords; // Desestructuración del
            objeto coords

            // Añadir un marcador en la posición del usuario
            L.marker([latitude, longitude])
                .addTo(mapa)
                .bindPopup("Tu ubicación actual");

            if (msgGeo) msgGeo.textContent = "Se ha detectado tu ubicación
            aproximada.";
        },
        // Callback de ERROR (si rechaza o falla)
        (err) => {
            if (!msgGeo) return;
            // err.code es un número que indica el tipo de error
            if (err.code === 1) msgGeo.textContent = "Has denegado el acceso a tu
            ubicación.";
            else if (err.code === 2) msgGeo.textContent = "No se pudo determinar
            tu posición.";
            else if (err.code === 3) msgGeo.textContent = "Se agotó el tiempo para
            obtener tu ubicación.";
            else msgGeo.textContent = "No se pudo obtener tu ubicación.";
        }
    );
};

```

typeof L === "undefined" — comprueba si la variable global **L** (que crea Leaflet) existe. Si el CDN de Leaflet no cargó (sin conexión, etc.), **L** no existiría y nuestro código daría error. Esta comprobación evita ese crash.

Desestructuración **const { latitude, longitude } = pos.coords**: Esto es equivalente a:

```

const latitude = pos.coords.latitude;
const longitude = pos.coords.longitude;

```

Pero más conciso. Extrae las propiedades del objeto directamente en variables.

navigator.geolocation.getCurrentPosition(éxito, error): Esta es la API de Geolocalización del navegador. Recibe dos funciones: una que se ejecuta si el usuario acepta compartir su ubicación, y otra si la rechaza o hay un error.

Códigos de error de geolocalización:

- **1 (PERMISSION_DENIED)** → el usuario dijo que no.

- 2 (`POSITION_UNAVAILABLE`) → el dispositivo no puede determinar la posición.
- 3 (`TIMEOUT`) → tardó demasiado en responder.

4.3 Sistema de Favoritos con `localStorage`

¿Por qué `localStorage` y no `sessionStorage`? Los favoritos deben persistir aunque el usuario cierre el navegador y vuelva días después. `localStorage` no tiene caducidad, persiste hasta que el usuario lo borra manualmente o el código lo elimina. `sessionStorage` se borraría al cerrar la pestaña.

¿Por qué `JSON.stringify` y `JSON.parse`? `localStorage` solo puede guardar **cadenas de texto**. Los favoritos son un array de objetos:

```
[
  { id: "abc123", name: "Harry Potter", house: "Gryffindor", ... },
  { id: "def456", name: "Hermione Granger", house: "Gryffindor", ... }
]
```

Esto no es un string. Para guardarlo:

```
// SIN stringify (MAL): guarda "[object Object],[object Object]" → inútil
localStorage.setItem("hp_favoritos", [{ id: "abc123" }]);

// CON stringify (BIEN): guarda '[{"id":"abc123","name":"Harry Potter",...}]'
localStorage.setItem("hp_favoritos", JSON.stringify([{ id: "abc123" }]));
```

Y para leerlo:

```
// SIN parse: devuelve el string '[{"id":"abc123",...}]' → no podemos usar
// .filter() etc.
localStorage.getItem("hp_favoritos");

// CON parse: devuelve el array de objetos real
JSON.parse(localStorage.getItem("hp_favoritos"));
```

Los helpers de `localStorage`:

```
// Leer todos los favoritos guardados
const getFavoritos = () => JSON.parse(localStorage.getItem("hp_favoritos")) || [];
//
// ^^^^^^
//
// Si no hay nada guardado, getItem devuelve null.
// JSON.parse(null) = null. null || [] = [].
// Así siempre devolvemos un array (nunca null).

// Guardar el array de favoritos
```

```
const saveFavoritos = (favs) => localStorage.setItem("hp_favoritos",
JSON.stringify(favs));

// Comprobar si un personaje ya es favorito (por su ID único)
const esFavorito = (id) => getFavoritos().some((f) => f.id === id);
// .some() recorre el array y devuelve true si AL MENOS UN elemento cumple la
condición
// Equivale a "¿hay algún favorito cuyo id coincida con este id?"
```

Toggle de favorito:

```
const toggleFavorito = (personaje) => {
  let favs = getFavoritos(); // Leer estado actual

  if (esFavorito(personaje.id)) {
    // Ya es favorito → quitarlo
    favs = favs.filter((f) => f.id !== personaje.id);
    // .filter() devuelve TODOS los favoritos EXCEPTO el que tiene ese id
  } else {
    // No es favorito → añadirlo
    favs.push({
      id: personaje.id,
      name: personaje.name,
      house: personaje.house || "Sin casa",
      species: personaje.species || "",
      image: personaje.image || "",
    });
    // Solo guardamos los campos que necesitamos (no todo el objeto de la API)
  }

  saveFavoritos(favs); // Guardar cambios
};
```

`favs.filter((f) => f.id !== personaje.id)` — ¿por qué `filter` para eliminar? `filter` devuelve un nuevo array con los elementos que cumplen la condición `f.id !== personaje.id`. Los elementos con el mismo id (el que queremos eliminar) no cumplen la condición, así que quedan fuera. Es la forma idiomática de eliminar un elemento de un array por condición en JavaScript.

El botón cambia de apariencia:

```
const actualizarBotonFavorito = (btn, personaje) => {
  if (esFavorito(personaje.id)) {
    btn.textContent = "★ Quitar favorito"; // Estrella rellena
    btn.className = "btn btn-sm btn-warning"; // Botón amarillo sólido
  } else {
    btn.textContent = "☆ Marcar como favorito"; // Estrella vacía
    btn.className = "btn btn-sm btn-outline-warning"; // Botón amarillo con
borde
  }
};
```

```
}  
};
```

Se llama a esta función cada vez que el usuario pulsa el botón para reflejar el estado actual.

5. Ampliación – Frases por personaje

Archivo: `dist/frases.html` + `js/frases.js`

5.1 Carga asíncrona de personajes por casa

```
const API_CASA_URL = "https://hp-api.onrender.com/api/characters/house/";  
const API_TODOS_URL = "https://hp-api.onrender.com/api/characters";  
  
const cargarPersonajesDeCasa = async () => {  
  const valor = selectCasa.value; // Ej: "Gryffindor", "Slytherin", "SIN_CASA"  
  
  if (valor === "SIN_CASA") {  
    // La API no tiene endpoint para "sin casa", así que pedimos TODOS y  
    filtramos  
    const res = await fetch(API_TODOS_URL);  
    const todos = await res.json();  
    personajes = todos.filter((p) => !p.house);  
    // !p.house → true si p.house es "", null, undefined (valores "falsy")  
  } else {  
    // Endpoint específico: .toLowerCase() porque la API espera minúsculas  
    const res = await fetch(API_CASA_URL + valor.toLowerCase());  
    // Para "Gryffindor" → "https://hp-api.onrender.com/api/characters/house/gryffindor"  
    personajes = await res.json();  
  }  
};
```

¿Por qué el value de "Sin casa especificada" es "SIN_CASA" y no ""? El primer `<option>` del select es el placeholder deshabilitado con `value=""`. Si "Sin casa" también tuviese `value=""`, serían indistinguibles en el JavaScript. Con un valor especial "SIN_CASA", podemos detectar cuál eligió el usuario.

¿Por qué esta petición es válida como "asíncrona"? Se hace en el mismo momento que el usuario elige la casa (`selectCasa.addEventListener("change", ...)`). No se carga anticipadamente. Cada vez que cambias de casa, se hace una nueva petición a la API.

5.2 El modal – abrir y cargar datos del personaje de forma asíncrona

Este es el punto más importante de la Ampliación. El enunciado dice explícitamente:

"La información del personaje deberá obtenerse de forma asíncrona **en ese momento** [al pulsar 'Añadir frases']. No se considerará válida una solución en la que se cargue toda la información

detallada al realizar la selección inicial de la casa."

Esto significa: cuando el usuario pulsa "Añadir frases", en ese preciso momento se hace la petición a la API para ese personaje concreto.

```
const API_PERSONAJE_URL = "https://hp-api.onrender.com/api/character/";

const abrirModal = async (personajeId) => {
  // 1. Mostrar el modal inmediatamente con un spinner (feedback visual)
  if (!modalInstance) {
    modalInstance = new bootstrap.Modal(document.getElementById("modal-
frases"));
  }
  modalInstance.show();

  // Resetear el estado del modal (por si venía de otro personaje)
  infoDiv.innerHTML = "";
  infoDiv.appendChild(loaderModal); // Mostrar spinner de carga
  loaderModal.classList.remove("d-none");
  titulo.textContent = "Cargando personaje...";

  try {
    // 2. Petición ASYNC al endpoint del personaje específico
    const res = await fetch(API_PERSONAJE_URL + personajeId);
    // URL resultante: "https://hp-api.onrender.com/api/character/9e3f7ce4-
b9a7-..."

    if (!res.ok) throw new Error(`Error HTTP ${res.status}`);

    const data = await res.json();
    // La API devuelve un array con UN solo elemento, así que tomamos data[0]
    const p = Array.isArray(data) ? data[0] : data;
    // Array.isArray() comprueba si es un array. Si no lo es, lo usamos
    directamente.

    personajeSeleccionado = p; // Guardar referencia global para usarla al
    guardar

    // 3. Ocultar spinner y mostrar info del personaje
    loaderModal.classList.add("d-none");
    titulo.textContent = `Frases de ${p.name}`;

    infoDiv.innerHTML = `
      ${p.image ? `` : ""}

      <div>
        <strong class="fs-5">${p.name}</strong><br>
        <small class="text-muted">
          ${p.house || "Sin casa"} · ${p.species || ""}
          ${p.yearOfBirth ? " · Nacido en " + p.yearOfBirth : ""}
        </small>
    `;
  }
}
```

```

        </div>
    `;
    // Operador ternario: condición ? "si verdadero" : "si falso"
    // p.yearOfBirth ? "..." : "" → solo muestra el año si existe

    // 4. Mostrar frases ya guardadas para este personaje
    renderFrasesEnModal(p.id);

    } catch (error) {
        loaderModal.classList.add("d-none");
        infoDiv.innerHTML = `<span class="text-danger">Error: ${error.message}
</span>`;
    }
};

```

new bootstrap.Modal(elemento) — así se crea una instancia del modal de Bootstrap desde JavaScript. El objeto **bootstrap** es el global que expone Bootstrap 5 cuando se carga su JS. **.show()** lo muestra.

personajeSeleccionado = p — variable global que guarda el personaje actualmente en el modal. La función de guardar frases la necesita para saber a qué personaje pertenece la frase.

5.3 Validaciones al guardar una frase

```

btnGuardar.addEventListener("click", () => {
    if (!personajeSeleccionado) return; // Seguridad: si no hay personaje, no
    hacer nada

    const frase = inputFrase.value.trim(); // Trim elimina espacios al inicio y
    final
    errorFrase.textContent = ""; // Limpiar errores anteriores

    // Validación 1: frase vacía o solo espacios
    if (!frase) {
        errorFrase.textContent = "La frase no puede estar vacía ni contener solo
    espacios.";
        return; // Salir de la función sin guardar
    }

    // Validación 2: frase duplicada para este personaje
    const frasesExistentes = getFrasesPersonaje(personajeSeleccionado.id);
    const duplicada = frasesExistentes.some(
        (f) => f.frase.toLowerCase() === frase.toLowerCase()
    );
    // .some() → devuelve true si alguna frase existente coincide (sin distinguir
    mayúsculas)

    if (duplicada) {
        errorFrase.textContent = "Esta frase ya está guardada para este
    personaje.";
        return;
    }

```

```

    }

    // Guardar si pasa las validaciones
    guardarFrase(personajeSeleccionado.id, personajeSeleccionado.name, frase);
    inputFrase.value = ""; // Limpiar el input para facilitar escribir otra
    renderFrasesEnModal(personajeSeleccionado.id); // Actualizar lista en el modal
    renderFrasesGuardadas(); // Actualizar la Sección 3
  });

```

¿Por qué `!frase` es suficiente para detectar vacío? Después de `.trim()`, si el usuario escribió solo espacios, `frase` será `""`. Una cadena vacía es "falsy" en JavaScript (`!""` es `true`). Así, `!frase` detecta tanto `""` como `" "` (porque `.trim()` lo convierte en `""`).

¿Por qué `.toLowerCase()` en la comprobación de duplicados? Para que "Expecto patronum" y "expecto Patronum" se consideren la misma frase. Sin esto, el mismo texto con diferente capitalización podría guardarse dos veces.

5.4 Estructura de datos en localStorage (frases)

```

const KEY_FRASES = "hp_frases"; // Clave en localStorage

const guardarFrase = (personajeId, personajeName, frase) => {
  const todas = todasLasFrases(); // Leer todas las frases actuales

  todas.push({
    personajeId, // ID único del personaje (de la API)
    personajeName, // Nombre del personaje (para mostrarlo)
    frase, // El texto de la frase
    fecha: new Date().toISOString() // Fecha y hora de inserción en formato
    ISO // Ejemplo: "2025-05-03T10:30:00.000Z"
  });

  localStorage.setItem(KEY_FRASES, JSON.stringify(todas)); // Guardar
};

```

`new Date().toISOString()` — genera la fecha y hora actual en formato ISO 8601 estándar: "2025-05-03T10:30:00.000Z". Se usa este formato porque:

1. Es un string que se puede guardar en localStorage.
2. Se puede comparar matemáticamente: `new Date("2025-05-03") > new Date("2025-01-01")` es `true`.
3. Es convertible de vuelta: `new Date("2025-05-03T10:30:00.000Z")` crea un objeto Date.

personajeId, (shorthand property) — cuando la variable y la propiedad tienen el mismo nombre, en lugar de `personajeId: personajeId` se puede escribir solo `personajeId`. Es sintaxis moderna de ES6.

5.5 Ordenación de frases por fecha

```
const renderFrasesGuardadas = () => {
  const orden = document.getElementById("orden-frases").value; // "asc" o "desc"

  const todas = todasLasFrases().sort((a, b) => {
    // Convertir los strings ISO a objetos Date para comparar
    const diff = new Date(a.fecha) - new Date(b.fecha);
    // Si a es más antigua que b: diff < 0 (número negativo)
    // Si a es más reciente que b: diff > 0 (número positivo)

    return orden === "desc" ? -diff : diff;
    // Para orden descendente (más reciente primero): invertir el signo
    // Para orden ascendente (más antiguo primero): mantener el signo
  });
};
```

¿Cómo funciona `sort((a, b) => a - b)`? `Array.sort` ordena el array modificándolo. La función comparadora recibe dos elementos (`a` y `b`):

- Si devuelve negativo → `a` va antes que `b`.
- Si devuelve positivo → `b` va antes que `a`.
- Si devuelve 0 → el orden se mantiene.

`new Date("2025-05-03") - new Date("2025-01-01")` devuelve la diferencia en milisegundos (un número positivo). Así `sort((a, b) => new Date(a.fecha) - new Date(b.fecha))` ordena de más antigua a más reciente. Negando el resultado (`-diff`) se invierte el orden.

6. Ampliación – Búsqueda avanzada por año

Archivo: `dist/busqueda-avanzada.html` + `js/busqueda-avanzada.js`

6.1 Las tres validaciones del formulario

```
const buscarPorAnho = async () => {
  const anhoIni = document.getElementById("anho-inicio").value.trim();
  const anhoFin = document.getElementById("anho-fin").value.trim();

  // Validación 1: campos obligatorios
  if (!anhoIni || !anhoFin) {
    mostrarError(zonaErrores, "Ambos campos son obligatorios.");
    return;
  }
  // !anhoIni es true si el campo está vacío (string vacío es falsy)

  const ini = parseInt(anhoIni, 10); // Convertir string "1980" al número 1980
  const fin = parseInt(anhoFin, 10); // El segundo parámetro 10 = base decimal
```

```
// Validación 2: valores numéricos válidos
if (isNaN(ini) || isNaN(fin)) {
    mostrarError(zonaErrores, "Los años deben ser valores numéricos válidos.");
    return;
}
// parseInt("abc") → NaN (Not a Number)
// parseInt("") → NaN
// isNaN(NaN) → true

// Validación 3: rango coherente
if (ini > fin) {
    mostrarError(zonaErrores, "El año inicial no puede ser mayor que el año final.");
    return;
}

// Si pasa las tres validaciones, buscar...
};
```

`parseInt(valor, 10)` — convierte un string a número entero. El `10` es la base (decimal). Sin él, podría interpretar "010" como octal en navegadores antiguos. Con `10`, siempre es decimal.

`isNaN()` — "is Not a Number". Devuelve `true` si el valor no es un número válido. `parseInt("abc", 10)` devuelve `NaN`, y `isNaN(NaN)` devuelve `true`.

6.2 Filtrado por `yearOfBirth`

```
const filtrados = datos.filter((p) => {
    const año = parseInt(p.yearOfBirth, 10);
    // p.yearOfBirth puede ser: 1980, "1980", "", null, undefined
    // parseInt(1980) → 1980
    // parseInt("1980") → 1980
    // parseInt("") → NaN
    // parseInt(null) → NaN
    // parseInt(undefined) → NaN

    return !isNaN(año) && año >= ini && año <= fin;
    // Condición 1: tiene un año válido (!isNaN)
    // Condición 2: el año está dentro del rango (>= ini y <= fin, extremos incluidos)
    // Si alguna condición falla, filter excluye ese personaje
});
```

Los personajes sin año de nacimiento (`yearOfBirth: ""` o `null`) son automáticamente excluidos porque `parseInt(null) = NaN` y `isNaN(NaN) = false`.

6.3 Agrupación por casa y acordeón Bootstrap

Paso 1: Crear el objeto de grupos

```
const grupos = {};  
// grupos es un objeto vacío que usaremos como diccionario: { "Casa": [personaje1,  
...] }  
  
personajes.forEach((p) => {  
  const casa = p.house || "Sin casa";  
  // Si p.house es "" o null → "Sin casa"  
  
  if (!grupos[casa]) grupos[casa] = [];  
  // Si todavía no existe el grupo para esta casa, créalo (array vacío)  
  
  grupos[casa].push(p);  
  // Añadir el personaje al grupo correspondiente  
});  
  
// Resultado ejemplo:  
// grupos = {  
//   "Gryffindor": [Harry, Hermione, Ron, ...],  
//   "Slytherin": [Draco, Snape, ...],  
//   "Sin casa": [Nearly Headless Nick, ...]  
// }
```

Paso 2: Ordenar las casas (principales primero)

```
const CASAS_ORDEN = ["Gryffindor", "Slytherin", "Hufflepuff", "Ravenclaw"];  
  
const casasOrdenadas = [  
  ...CASAS_ORDEN.filter((c) => grupos[c]),  
  // De las 4 casas principales, solo las que tienen personajes en los  
  // resultados  
  
  ...Object.keys(grupos).filter((c) => !CASAS_ORDEN.includes(c)),  
  // El resto de casas (incluida "Sin casa") que no están en CASAS_ORDEN  
];  
  
// Ejemplo si solo hay personajes de Gryffindor y Sin casa:  
// casasOrdenadas = ["Gryffindor", "Sin casa"]
```

Object.keys(grupos) — devuelve un array con los nombres de todas las propiedades del objeto: ["Gryffindor", "Sin casa"].

Spread operator ... — "desempaqueta" un array dentro de otro. [...arr1, ...arr2] es equivalente a arr1.concat(arr2).

Paso 3: Generar el acordeón dinámicamente

```
casasOrdenadas.forEach((casa, idx) => {
  // idx es el índice: 0, 1, 2... → necesario para IDs únicos

  const vivos = personajesDeCasa.filter((p) => p.alive === true);
  const muertos = personajesDeCasa.filter((p) => p.alive !== true);
  // === true → comparación estricta. Solo cuenta como vivo si p.alive es
  exactamente true
  // !== true → cualquier otro valor (false, null, undefined) → muerto

  const item = document.createElement("div");
  item.className = "accordion-item";
  item.innerHTML = `
    <h2 class="accordion-header" id="heading-${idx}">
      <button class="accordion-button collapsed" type="button"
        data-bs-toggle="collapse" data-bs-target="#collapse-${idx}">
        ${casa}
        <span class="badge bg-secondary ms-2">${personajesDeCasa.length}
        personaje(s)</span>
      </button>
    </h2>
    <div id="collapse-${idx}" class="accordion-collapse collapse">
      <div class="accordion-body">
        ${renderGrupoVidaMuerte(vivos, muertos)}
      </div>
    </div>
  `;
});
```

¿Cómo funciona el acordeón de Bootstrap? Bootstrap 5 usa atributos HTML especiales para el comportamiento interactivo:

- `data-bs-toggle="collapse"` → indica que este botón abre/cierra un panel
- `data-bs-target="#collapse-0"` → el id del panel que controla
- `class="accordion-collapse collapse"` → el panel empieza cerrado (sin `show`)
- Bootstrap gestiona automáticamente añadir/quitar la clase `show` al hacer click

Los **IDs dinámicos** (`heading-${idx}`, `collapse-${idx}`) son esenciales: si todos los paneles tuviesen el mismo id, el acordeón no funcionaría.

p.alive === true (triple igual) En la API, `p.alive` puede ser `true`, `false` o incluso estar ausente (`undefined`). Con `=== true` (comparación estricta de tipo y valor), solo cuentan como vivos los que explícitamente tienen `true`. Con `== true` (comparación laxa), `1 == true` también sería verdadero (y no queremos eso).

7. Gestión de errores: los 4 tipos

El enunciado pide diferenciar exactamente estos 4 tipos de error. Esta es la tabla y cómo se maneja cada uno:

Error 1: Error de red (fetch lanza una excepción)

Cuándo ocurre: Sin conexión a internet, el servidor de la API está caído, la URL no existe.

Cómo se detecta: `fetch` lanza un `TypeError` (no devuelve una respuesta, lanza directamente una excepción).

```
try {
  const res = await fetch("https://hp-api.onrender.com/api/characters");
  // Si no hay internet, esto lanza: TypeError: Failed to fetch
} catch (error) {
  const mensaje = error instanceof TypeError
    ? "Error de red: no se pudo conectar con la API."
    : `Error: ${error.message}`;
  mostrarError(zonaErrores, mensaje);
}
```

`error instanceof TypeError` — comprueba si el error es de tipo `TypeError`. Es la forma de distinguir "no llegué a conectarme" de "me conecté pero la API devolvió un error".

Error 2: Error HTTP (la API responde con código de error)

Cuándo ocurre: La API responde, pero con código 404 (no encontrado), 500 (error interno), etc.

Cómo se detecta: `fetch` NO lanza excepción en errores HTTP. Llega la respuesta, pero `respuesta.ok` es `false`.

```
const respuesta = await fetch(API_URL);

if (!respuesta.ok) {
  // respuesta.status: número del código HTTP (404, 500...)
  // respuesta.statusText: texto del error ("Not Found", "Internal Server Error"... )
  throw new Error(`Error HTTP ${respuesta.status}: ${respuesta.statusText}`);
}
// Si llegamos aquí, la respuesta es OK (código 200-299)
```

¿Por qué `fetch` no lanza error en 404/500? Porque técnicamente la comunicación fue exitosa: se envió una petición y se recibió una respuesta. Solo lanza excepción si no puede ni comunicarse (error de red). Por eso debemos comprobar `respuesta.ok` manualmente.

Error 3: Búsquedas sin resultados (array vacío)

Cuándo ocurre: La API responde correctamente pero no hay personajes que coincidan.

Cómo se detecta: El array filtrado tiene longitud 0.

```
const filtrados = datos.filter((p) => p.name.toLowerCase().includes(texto));

if (filtrados.length === 0) {
  mostrarError(zonaErrores, `No se encontraron personajes con el nombre
```



```
"${texto}").`);  
    return; // No intentar crear la tabla  
}
```

Error 4: Error de usuario (input vacío)

Cuándo ocurre: El usuario pulsa Buscar sin escribir nada.

Cómo se detecta: Después de `.trim()`, el string es `""`.

```
const texto = input.value.trim().toLowerCase();  
  
if (texto === "") {  
    if (esSubmit) { // Solo mostrar error si es click en el botón, no en tiempo  
        real  
            mostrarError(zonaErrores, "El campo de búsqueda no puede estar vacío.");  
    }  
    return;  
}
```

Cómo se muestra el error (Bootstrap Alert)

```
const mostrarError = (contenedor, mensaje) => {  
    const div = document.createElement("div");  
    div.className = "alert alert-danger alert-dismissible fade show mt-2";  
    div.role = "alert";  
    div.innerHTML = `${mensaje}  
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-  
label="Cerrar">  
        </button>`;  
    contenedor.appendChild(div);  
};
```

Clases Bootstrap del alert:

- `alert` → estilo base del aviso
- `alert-danger` → fondo rojo (para errores)
- `alert-dismissible` → espacio para el botón de cerrar
- `fade show` → animación de aparición suave
- `data-bs-dismiss="alert"` → Bootstrap maneja automáticamente el click para cerrar/eliminar el alert

8. Preguntas trampa del profesor

P: ¿Por qué la aplicación es de una sola página y no tiene varias? R: El enunciado lo exige explícitamente: "El sitio web constará de las siguientes secciones, todas visibles en la misma página." Las 4 secciones están en `index.html` separadas por `<hr>`. El navbar usa anclas (`#id`) para navegar a cada sección con scroll suave. Las

dos páginas adicionales de la Ampliación son nuevas páginas porque el enunciado dice "nuevas opciones que llevarán a una nueva página".

P: ¿Cuál es la diferencia entre `localStorage` y `sessionStorage`? ¿Por qué usas cada uno? R:

	<code>localStorage</code>	<code>sessionStorage</code>
Duración	Indefinida (hasta borrarlo manualmente)	Solo durante la sesión (se borra al cerrar la pestaña)
Scope	Compartido entre pestañas del mismo origen	Solo la pestaña actual
Uso en el proyecto	Favoritos y frases (deben persistir)	Aviso de cookies (basta con aceptar una vez por sesión)

P: ¿Por qué usas `async/await` y no `.then().catch()`? R: Ambas formas son equivalentes y válidas.

`async/await` es más legible porque el código se lee de arriba abajo como si fuera síncrono.

`.then().catch()` crea cadenas de callbacks anidadas que se hacen difíciles de leer. El enunciado pide "AJAX moderno con `fetch` + `async/await`".

P: ¿Qué ocurre si la API está caída cuando abre la página? R: El `try/catch` captura el error. Si `fetch` lanza un `TypeError` (sin conexión), se detecta con `error instanceof TypeError` y se muestra "Error de red: no se pudo conectar con la API." en la zona de errores del buscador. El spinner de la sección Bienvenida cambia a un mensaje de error. La página sigue funcionando para el resto de funcionalidades (favoritos guardados, frases guardadas) porque esas usan `localStorage`, no la API.

P: ¿Por qué `!respuesta.ok` y no `respuesta.status !== 200`? R: `respuesta.ok` es `true` para cualquier código HTTP entre 200 y 299 (todos los códigos de éxito). Si comprobáramos solo `=== 200`, fallaría para respuestas como `201 (Created)` o `204 (No Content)` que también son válidas. `ok` es el way estándar de comprobar éxito HTTP con `fetch`.

P: ¿Por qué guardas un subconjunto de datos del personaje en favoritos y no el objeto entero? R: El objeto completo de la API tiene muchos campos que no necesitamos (`patronus`, `yearOfBirth`, `ancestry`, `wand...`). Guardar solo `id`, `name`, `house`, `species`, `image` ahorra espacio en `localStorage` y deja claro qué datos vamos a usar. `localStorage` tiene un límite de ~5MB por dominio.

P: ¿Qué es `|| []` en `JSON.parse(localStorage.getItem("hp_favoritos")) || []`? R:

`localStorage.getItem` devuelve `null` si la clave no existe (primera vez que se usa la app).

`JSON.parse(null)` devuelve `null`. Hacer `null.filter(...)` daría un error de JavaScript ("Cannot read properties of null"). El `|| []` usa el operador OR lógico: si la expresión de la izquierda es "falsy" (`null`, `undefined`, `0`, `""`), devuelve la de la derecha (`[]`). Así siempre tenemos un array vacío como valor por defecto.

P: ¿Por qué no puedes buscar por nombre directamente en la API? R: La HP-API no tiene un endpoint de búsqueda por nombre. Solo devuelve todos los personajes, o todos los de una casa. Por eso el enunciado

explica que hay dos opciones válidas: cargar todos al inicio y filtrar localmente (Opción 1, que es lo que hacemos), o hacer una nueva petición por cada búsqueda (Opción 2). Elegimos la Opción 1 porque hace menos peticiones a la API y es más eficiente.

P: ¿Qué es el Bootstrap `accordion` y cómo funciona sin escribir JavaScript? R: El acordeón de Bootstrap funciona únicamente con atributos HTML (`data-bs-toggle`, `data-bs-target`). No necesita JavaScript propio porque Bootstrap incluye los listeners internamente cuando carga su JS. Los paneles se abren/cierran añadiendo y quitando la clase CSS `show`. Lo que Sí escribimos en JavaScript es la generación dinámica del HTML del acordeón con los datos de la API, porque no sabemos de antemano cuántas casas habrá en los resultados.

P: ¿Por qué en la búsqueda avanzada `p.alive !== true` incluye a los personajes con `alive = false` Y a los que no tienen el campo? R: `!== true` es "diferente de `true` en tipo Y valor". Entonces:

- `false !== true` → `true` → se considera muerto ✓
- `null !== true` → `true` → se considera muerto ✓
- `undefined !== true` → `true` → se considera muerto ✓

Si usáramos `p.alive === false`, los personajes con `alive: null` o sin el campo no entrarían en ningún grupo. Con `!== true` nos aseguramos de que TODOS los personajes quedan clasificados en uno de los dos grupos.

P: ¿Por qué hay que regenerar el HTML del acordeón cada vez que se hace una búsqueda nueva? R: `resultados.innerHTML = ""` borra todo el contenido previo antes de insertar los nuevos resultados. Si no lo hiciéramos, los resultados anteriores y los nuevos se acumularían visualmente. Es una práctica común: limpiar el contenedor antes de renderizar nuevos datos.

Proyecto: *dwec_t06p01_arroyo_mikel* · DWECC 2ª Ordinaria · Arroyo Mikel 25-26